ENGINEERING REFERENCE GUIDE

k6 Load Test Executors

Executors control how Virtual Users (VUs) and iterations are scheduled and managed in k6. Selecting the proper executor is vital for accurate capacity planning, throughput evaluation, and realistic user traffic simulation.

1. Constant VUs (constant-vus)

Need: Steady concurrent baseline load **Time-Driven**

Maintains an exact, unwavering number of active users continuously for a set duration. VUs instantly restart iterations upon completion.

```
import http from 'k6/http';
export const options = {
  scenarios: {
    steady: { executor: 'constant-vus', vus: 5, duration: '20s' },
  },
};
export default function() { http.get('https://test-api.k6.io/public/crocodiles/'); }
```

2. Ramping VUs (ramping-vus)

Need: Stress testing & traffic peaks **Stage-Driven**

Simulates growing or shrinking concurrent active populations across customizable time stages to observe scaling behavior.

```
export const options = {
  scenarios: {
    ramp: {
      executor: 'ramping-vus',
      stages: [
        { duration: '10s', target: 5 },
        { duration: '20s', target: 0 },
      ],
    },
  },
};
```

Iteration-Driven & Throughput Executors

3. Shared Iterations (shared-iterations)

Need: Finite batch processing tasks **Work-Driven**

A collective pool of VUs shares a total predefined number of work iterations until completion.
Here in this example:
- load simulated total 50 request sent by total 5 VUs.

```
export const options = {
  scenarios: {
    batch: { executor: 'shared-iterations', vus: 5, iterations: 50, maxDuration: '30s' },
  },
};
```

4. Per-VU Iterations (per-vu-iterations)

Need: Identical unit workload distribution **Isolated Work**

Guarantees every individual VU executes an exact, equal amount of discrete task iterations.
Here in this example:
- k6 spins up 3 VUs and 10 iterations each. So total 3*10 requests are sent.

```
export const options = {
  scenarios: {
    exact: { executor: 'per-vu-iterations', vus: 3, iterations: 10 },
  },
};
```

5. Constant Arrival Rate (constant-arrival-rate)

Need: True system throughput & RPS capacity **Rate-Driven**

Forces strict requests-per-second regardless of server latency, exposing backend bottleneck limits.
Here in this example it tells:
- 10 request per seconds will be sent for a duration of 15 seconds
- k6 will maintain this RPS using 5-20 VUs.

```
export const options = {
  scenarios: {
    throughput: {
      executor: 'constant-arrival-rate',
      rate: 10, timeUnit: '1s', duration: '15s',
      preAllocatedVUs: 5, maxVUs: 20,
    },
  },
};
```